Product Requirements Document

Chitra AI

Generative AI Image Generation Application

Version: 1.2 **Date:** August 25, 2026 **Status:** Development Ready

1. Product Overview

Chitra AI is a full stack generative AI image generation application that allows users to create images from natural language prompts. The application provides a focused workflow for writing prompts, configuring image generation options, generating images, previewing results, downloading images, and browsing generation history.

Chitra AI will use React for the frontend, Django for the backend API, PostgreSQL for persistent data, and Hugging Face Inference Providers for AI image generation.

Product name: Chitra AI
Product category: Generative AI Image Generation
Primary purpose: Generate and manage AI created images from natural language prompts.

2. Product Goals

• Allow users to generate images using natural language prompts.

• Demonstrate integration with a real GenAI image generation API.

• Keep Hugging Face API credentials secure on the backend.

• Provide a simple and intuitive image generation experience.

• Allow users to download generated images.

• Maintain a persistent generation history.

• Demonstrate production quality loading and error handling.

• Deploy Chitra AI to a production environment.

• Teach a reusable UI and UX system rather than only an API integration.

3. Non Goals

• Advanced image editing

• Image to image generation

• Video generation

• Fine tuning custom AI models

• Social sharing

• Marketplace functionality

• Subscription billing

• Complex user roles

• Enterprise administration

4. Target Users

Primary User

Students, developers, and AI enthusiasts who want to experiment with generative image creation.

Secondary Users

Designers, content creators, marketers, educators, and developers who need quick AI generated visual assets.

5. Technology Stack

Frontend

React. Handles the Chitra AI interface, prompt input, generation controls, API communication, loading states, error states, image preview, history, and downloads.

Backend

Django + Django REST Framework. Handles API endpoints, validation, Hugging Face integration, credential protection, metadata management, and business logic.

Database

PostgreSQL. Stores generated image records, prompts, generation configuration, image URLs, provider information, and timestamps.

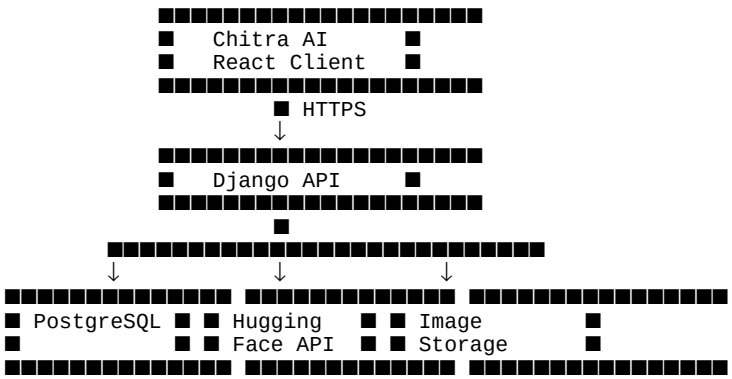
AI Provider

Hugging Face Inference Providers. The first implementation should use an available text to image model such as FLUX. Provider specific code should be isolated behind a service layer.

Deployment

```
Chitra AI React frontend → Vercel
Chitra AI Django backend → Cloud deployment
PostgreSQL → Managed PostgreSQL
Generated Images → Object Storage
```

6. High Level Architecture



7. Core User Journey

```
Open Chitra AI
      ↓
Enter Image Prompt
      ↓
Select Image Size
      ↓
Select Image Quality
      ↓
```

```
Click Generate
       ↓
Display Loading State
       ↓
Django Sends Request
       ↓
Hugging Face Generates Image
       ↓
Django Saves Metadata
       ↓
Return Image
       ↓
Display Generated Image
       ↓
Download or View History
```

8. Functional Requirements

FR 01: Prompt Input

Provide a text area for image prompts. Prevent empty prompts and provide guidance for effective prompts.

FR 02: Prompt Validation

Validate prompt presence, length, supported parameters, and request structure on the backend.

FR 03: Image Size

Provide supported image sizes such as 1024 × 1024, 1024 × 1536, and 1536 × 1024, subject to the selected model.

FR 04: Image Quality

Expose quality options supported by the selected provider or map them appropriately.

FR 05: Image Generation

React submits to Django, Django validates and calls Hugging Face, saves metadata, and returns image information.

FR 06: Loading State

Disable duplicate submissions and provide clear generation feedback.

FR 07: Generated Image Display

Display image, prompt, dimensions, quality, timestamp, and download action.

FR 08: Download Image

Allow users to download the generated image using a meaningful filename.

FR 09: Image History

Provide a gallery with thumbnail, prompt, size, creation date, and download action.

FR 10: Delete History

Allow deletion with confirmation before destructive action.

FR 11: Error Handling

Handle validation, network, provider, rate limit, authentication, timeout, storage, and database errors.

FR 12: Provider Abstraction

Use a service abstraction so additional AI providers can be added without rewriting the application.

9. UI/UX Design System and Interaction Specification

Chitra AI must define a consistent visual and interaction system. UI and UX are first class product requirements and must not be treated as decoration added after implementation.

9.1 Design Direction

The interface should feel modern, focused, trustworthy, and product oriented. The generated image should remain the visual focus. Avoid generic AI SaaS patterns such as excessive gradients, unnecessary glassmorphism, decorative AI illustrations, and visually noisy layouts.

9.2 Design System

Define and document a reusable Chitra AI design system covering:

• Brand color palette and semantic colors

• Typography family, sizes, weights, and line heights

• Spacing scale

• Border radius

• Elevation and shadows

• Iconography

• Buttons

• Form controls

• Cards

• Dialogs and modals

• Notifications and status messages

9.3 Layout System

```
Chitra AI Application Shell
■■■ Header
■■■ Main Content
■   ■■■ Generator
■   ■■■ Result
■■■ History
```

The layout must be responsive and maintain clear visual hierarchy across mobile, tablet, and desktop.

9.4 Generator UX

```
Prompt
   ↓
Generation Settings
   ↓
Generate
   ↓
Generation State
   ↓
Result
   ↓
Download / Regenerate
```

The Generate action is the primary call to action. The interface should make the main workflow immediately understandable without requiring instructions.

9.5 Empty State

Before the first generation, the result area should communicate what the user can do instead of showing an unexplained empty container.

Example: Describe an image and your generated result will appear here.

9.6 Loading UX

During generation, preserve the entered prompt and selected settings. Disable duplicate submissions and provide clear feedback.

```
Generating image
Creating your visual...
```

9.7 Error UX

Errors should appear close to the action that caused them. User facing messages should be understandable and must not expose raw provider errors.

```
Could not generate the image.

Please try again.
```

The backend should log useful diagnostic information separately from the user facing message.

9.8 History UX

The history gallery should support thumbnail preview, full image preview, prompt visibility, download, delete, and regeneration where supported.

9.9 Accessibility

• Keyboard navigation

• Visible focus states

• Accessible labels

• Meaningful alt text

• Sufficient color contrast

• Screen reader friendly controls

• Reduced motion support

9.10 Component States

Every major interactive component should define:

```
Default
Hover
Focus
Active
Disabled
Loading
Success
Error
Empty
```

9.11 Responsive Behavior

Define responsive breakpoints and component behavior for mobile, tablet, and desktop. Controls must remain usable without horizontal scrolling. Image previews should scale without breaking the page layout.

9.12 UX Quality Principles

• Minimize cognitive load.

• Keep the primary action obvious.

• Give immediate feedback after user actions.

• Preserve user input during errors.

• Use progressive disclosure for advanced settings.

• Keep destructive actions visually distinct.

• Prefer clear language over technical API terminology.

• Maintain consistency across all screens.

10. API Requirements

POST /api/images/generate/

Generates an image.

```
{
  "prompt": "A futuristic city at sunset",
  "size": "1024x1024",
  "quality": "standard"
}
```

Example response:

```
{
  "id": 42,
  "prompt": "A futuristic city at sunset",
  "image_url": "https://example.com/image.png",
  "size": "1024x1024",
  "quality": "standard",
  "provider": "huggingface",
  "created_at": "2026-08-25T09:30:00Z"
}
```

GET /api/images/ returns generation history.

GET /api/images/{id}/ returns details for one generation.

DELETE /api/images/{id}/ deletes a generation.

11. Database Requirements

GeneratedImage

```
id
prompt
image_url
size
quality
provider
model
created_at
updated_at
user_id (when authentication is introduced)
```

12. Security Requirements

• Never expose the Hugging Face API token to the React client.

• Store secrets using environment variables.

• Use HTTPS in production.

• Restrict CORS.

• Validate backend requests.

• Use rate limiting.

• Never commit secrets to Git.

• Do not expose sensitive provider errors.

```
HF_TOKEN=hf_xxxxxxxxx
```

13. Prompt Engineering

Teach users to construct prompts around subject, environment, composition, lighting, style, mood, and additional details.

```
Subject
+
Environment
+
Composition
+
Lighting
+
Style
+
Mood
+
Additional details
```

14. UI Requirements

Main Page

```
Chitra AI

Describe your image
[ Prompt textarea ]

Image Size
[ 1024 × 1024 ]

Quality
[ Standard ]

[ Generate Image ]

Generated Image
[ Image ]
[ Download ]
```

History Page

```
Chitra AI Generation History

■■■■■■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■■■■■■■
■ Image     ■ ■ Image     ■ ■ Image     ■
■ Prompt    ■ ■ Prompt    ■ ■ Prompt    ■
■ Download  ■ ■ Download  ■ ■ Download  ■
■■■■■■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■■■■■■■
```

15. Responsive Requirements

Chitra AI must work on desktop, laptop, tablet, and mobile. The gallery should adapt its column count according to viewport width.

16. Performance Requirements

• Avoid unnecessary API requests.

• Prevent duplicate generation requests.

• Lazy load history thumbnails where appropriate.

• Optimize image delivery.

• Paginate large image histories.

• Provide appropriate loading feedback.

17. Observability

• API failures

• Generation failures

• Response latency

• Request volume

• Rate limit errors

• Server errors

Provider API credentials and sensitive request information must never be written to logs.

18. Testing Requirements

Frontend

• Prompt validation

• Form submission

• Loading state

• Successful generation

• Error state

• Image rendering

• Download interaction

• History rendering

Backend

• Image generation endpoint

• Request validation

• Hugging Face service

• Provider errors

• Database persistence

• Image history

• Delete operation

• Authentication if implemented

Integration

```
React
 ↓
Django
 ↓
Hugging Face
 ↓
PostgreSQL
 ↓
React
```

19. Environment Variables

```
DJANGO_SECRET_KEY=
DATABASE_URL=
```

```
HF_TOKEN=
HF_MODEL=
CORS_ALLOWED_ORIGINS=
```

20. Deployment Requirements

• Environment variables

• HTTPS

• CORS

• Production database

• Static files

• Media files

• Error logging

21. MVP Scope

✓ Chitra AI React interface

✓ Prompt input

✓ Image size selection

✓ Image generation

✓ Hugging Face integration

✓ Django API

✓ PostgreSQL

✓ Generated image preview

✓ Image download

✓ Loading states

✓ Error handling

✓ Image history

✓ Delete history

✓ Responsive design

✓ UI/UX design system

✓ Production deployment

22. Future Features

• User authentication

• User specific galleries

• Prompt templates

• Prompt enhancement

• Favorite images

• Image regeneration

• Image variations

• Image editing

• Image to image generation

• Multiple AI providers

• Generation credits

• Subscription plans

• Public image sharing

• Community gallery

• Advanced generation parameters

• AI generated prompt suggestions

23. Acceptance Criteria

1. A user can enter a prompt.

2. The frontend can submit the prompt to Django.

3. Django can securely communicate with Hugging Face.

4. Hugging Face can generate an image.

5. The generated image is displayed in React.

6. Generation metadata is stored in PostgreSQL.

7. Users can download generated images.

8. Users can view previous generations.

9. Users can delete history items.

10. Loading states work correctly.

11. API and validation errors are handled gracefully.

12. Hugging Face credentials are never exposed to the browser.

13. Chitra AI has a documented and consistent UI/UX system.

14. All major components define interactive and feedback states.

15. The application works on mobile and desktop.

16. The application can be deployed to production.

17. The deployed application works end to end.

24. Educational Modules

1. Project Overview

2. Setting Up React and Django

3. Understanding Generative Image APIs

4. Creating a Hugging Face Account and API Token

5. Integrating Hugging Face with Django

6. Designing the Chitra AI UI/UX System

7. Building the Image Generation UI

8. Writing Effective Image Prompts

9. Configuring Image Size and Quality

10. Displaying Generated Images

11. Downloading Generated Images

12. Managing Loading and Error States

13. Saving Generation Metadata

14. Building the Image History Gallery

15. Testing the Application

16. Deploying Chitra AI

25. Success Metrics

• Successful image generation rate

• API error rate

• Average generation response time

• Number of successful generations

• Number of images saved to history

• Successful download rate

• Application availability

• Student ability to explain the complete AI request flow

• Student ability to apply the Chitra AI design system consistently across screens

26. Final Product Definition

Chitra AI is a full stack GenAI image generation platform where a user writes a prompt, React sends it to Django, Django communicates securely with Hugging Face, the AI generated image is returned and persisted with its metadata, and React provides preview, download, and history functionality. The product includes a documented UI/UX system that defines its visual language, interaction states, responsive behavior, accessibility, and primary generation workflow.

```
User
 ↓
Writes a prompt
 ↓
Chitra AI React
 ↓
Django API
 ↓
Hugging Face
 ↓
AI generated image
 ↓
Django
 ↓
PostgreSQL + Image Storage
 ↓
Chitra AI React
 ↓
Preview
 ↓
Download
```